Technical Report

大氣垂直風場動態分析系統

(Dynamic Wind Profile Visualizer)

Albert Sheng 4/20/2026

1. 專案簡介與目標 (Detailed Introduction & Objectives)

1.1 研究背景與技術動機 (Background and Motivation)

在大氣科學與環境工程領域，行星邊界層（Planetary Boundary Layer, PBL）的動力學特性對於理解局部天氣模式、污染物擴散機制以及風力發電場的運作至關重要。垂直風場剖面（Vertical Wind Profiles）描述了風速與風向隨高度變化的連續狀態，是表徵邊界層結構的核心數據。傳統上，這些數據由多普勒雷射雷達（Doppler Lidar）或聲學雷達（SODAR）採集，生成的原始輸出通常為 .hpl 或類似的 ASCII 文字格式。

然而，在長期觀測與高頻率（如 10 分鐘一筆）的資料產出下，現有的分析流程面臨以下挑戰：

1. **計算延遲與 I/O 瓶頸**：ASCII 檔案的解析（Parsing）涉及大量的字符串掃描與浮點數轉換，當處理涉及數萬個檔案的多年份趨勢分析時，傳統的逐檔讀取方法會導致系統響應極其緩慢。
2. **資料孤島與碎片化**：原始數據往往缺乏結構化的元數據（Metadata）管理，時間戳與觀測參數分布在檔名與非結構化標頭中，增加了自動化處理的複雜度。
3. **視覺化維度缺失**：靜態的 2D 剖面圖難以捕捉大氣中轉瞬即逝的運動過程。例如，低空噴流（Low-Level Jets）的演變或埃克曼螺旋（Ekman Spiral）在夜間穩定層結下的旋轉特徵，需要具備時序動態感的視覺化工具才能被準確識別。

1.2 系統開發目標 (System Objectives)

本專案旨在建構一個整合後端高效數據轉換與前端互動分析的軟體系統，達成以下具體目標：

- **高效能數據架構轉換**：開發專用的 WindDataProcessor，將非結構化的 ASCII .hpl 數據遷移至高效能的 Apache Parquet 列式存儲格式。Parquet 格式具備強大的壓縮性能與預先編碼特性，目標是將數據讀取速度提升一個數量級以上。
- **物理模型準確映射**：系統需嚴格遵循氣象學座標系，確保風向向量的三角函數轉換（u/v 分量分解）在數學上與大氣運動方程式保持一致，並解決常見的數學座標與氣象座標（North-Up）之差異問題。

- **高流暢度動態分析平台**：基於 Python Streamlit 框架，利用 Plotly 的 Frames 渲染機制，開發一個支持 30 FPS 以上播放速率的互動式儀表板。該平台需允許分析人員在播放過程中即時調整觀測高度、風速標尺與向量比例。
- **工業級 UI/UX 實作**：透過 CSS 注入與前端佈局優化，解決複雜動畫圖表在跨設備、縮放環境下控制按鈕重疊或視覺模糊的問題，提供穩定且具專業感的操作介面。
- **穩健的環境適應性**：系統需具備良好的跨平台能力，透過 pathlib 模組解決 Windows 環境下常見的路徑分隔符與轉義字元警告問題，確保在不同的工程工作站上皆能穩定部署。

透過本系統的實施，研究人員將能從繁重的數據清洗工作中解脫，專注於捕捉大氣邊界層中細微的運動力學特徵，提升大氣觀測診斷的精確度與科學價值。

1.3 文件閱讀與系統適用性說明 (Operational Instructions & Scope)

為確保研究人員能準確解讀本報告與操作系統，閱覽本章節及後續內容時應遵循以下準則：

- **核心維度定義**：本系統主要針對 PBL 層之垂直風場進行視覺化優化，其物理引擎之設計假設為局部穩定大氣。
- **物理邊界限制**：所有向量場之計算均假設輸入數據已通過 WindDataProcessor 之信噪比（SNR）檢驗。
- **專業知識門檻**：閱覽者應具備基礎大氣物理知識與向量力學（Vector Mechanics）基礎，以準確理解第四章中關於座標轉換與長寬比修正（Aspect Ratio Fix）之邏輯。
- **軟體版本依賴**：本開發報告所述之功能皆基於 Python 3.10+ 環境，請確保相依庫版本與附錄所述一致。

2. 系統架構與技術棧 (System Architecture & Technology Stack)

2.1 系統設計哲學：模組化與關注點分離 (Design Philosophy)

本系統的開發核心基於「關注點分離」（Separation of Concerns, SoC）的工程原則。針對大氣觀測資料的高頻率與非結構化特性，系統架構被劃分為三個垂直層級：資料處理層（ETL Tier）、資料存儲層（Persistence Tier）以及互動分析層（Presentation Tier）。

這種分層設計確保了後端邏輯與前端介面的低耦合性。軟體工程上的考量在於，當感測器（如 Lidar 或 SODAR）的原始輸出格式發生變經時，僅需更新後端處理模組（processor.py），而無須變動前端動態展示邏輯（app.py）。選擇 Python 作為核心語言，主因在於其成熟的科學計算生態體系（PyData Ecosystem），能無縫銜接底層 C 語言優化的數值運算庫與高層級的 Web 框架。

2.2 後端引擎：高密度大氣資料的處理與轉換

大氣剖面資料通常具備極高的時間與空間解析度。在傳統 ASCII 文本處理模式下，每讀取一筆資料均需經過系統調用（System Call）與字串轉換，這對於工程分析而言效率極低。

- **Pandas 與 NumPy 的向量化運算**： 系統放棄了傳統的遞迴循環處理，轉而採用 NumPy 的向量化（Vectorization）操作。在處理風向向量的 u（緯向風）與 v（經向風）分量分解時，系統利用多維陣列運算直接作用於整個高度層，這在計算力學上能最大限度利用現代 CPU 的 SIMD 指令集。例如，針對單日 144 個時段、每個時段 100 層高度的資料（約 14,400 點），向量化運算能在毫秒級完成。

- **Pathlib 與穩健性路徑工程**： 針對機械與電力工程現場常用的 Windows 環境，系統棄用了不穩定的字串路徑拼接（os.path），全面導入 pathlib.Path 對象。這種物件導向的路徑處理方式，不僅自動解決了反斜線（Backslash）轉義問題，更確保了在不同掛載磁碟路徑（如 K: 或 J: 槽）下的路徑解析一致性，大幅降低了檔案 I/O 異常的風險。

- **Tkinter 的整合應用**： 為了提供直覺的工程操作體驗，後端引擎整合了 tkinter 的原生檔案對話框（filedialog）。這使得 WindDataProcessor 能在非網頁環境下獨立運行，方便工程人員進行批次資料預處理。

2.3 資料存儲層：Apache Parquet 列式存儲之應用

本系統捨棄了傳統的 CSV 或 SQL 存儲，轉而採用工業級的 **Apache Parquet** 格式。這是一項關鍵的技術決策，其考量點包括：

- **列式存儲（Columnar Storage）優勢**： 在風場分析中，我們往往僅需針對「風速」或「風向」單一屬性進行跨時間檢索。Parquet 格式允許系統僅讀取磁碟中相關列的二進位塊，而非加載整行無關資料。這將磁碟 I/O 的負載降低了約 60%-80%。

- **二進位壓縮與編碼**： 利用 Snappy 壓縮算法，Parquet 能在不損失浮點數精度（Floating-point precision）的前提下，將原始 .hpl 文本體積壓縮至原本的 10% 左右。對於長期觀測站點而言，這不僅節省存儲成本，更提升了緩存（Caching）的命中率。

- **強型別 Schema 保護**： Parquet 內置 Schema 定義，確保了時間戳、高度（float64）與向量數值在存儲與加載過程中的型別一致性，杜絕了文本解析中常見的空值（NaN）誤判與格式混亂問題。

2.4 前端框架：反應式 Web 架構（Reactive Framework）

前端展示層選用 **Streamlit** 與 **Plotly** 的組合，構建一個高效的單頁應用程式（SPA）。

- **Streamlit 的執行模型與 Session State**： Streamlit 採取自上而下的腳本式執行模型，這雖然簡化了開發，但在動畫播放時容易產生刷新頻率不穩定的問題。系統

透過 st.session_state 管理「播放狀態」(Playing Status）與「時間索引」(Time Index)，並結合 @st.cache_data 對已加載的二進位資料進行記憶體快取。這使得使用者在切換觀測日期時，系統無需重新從磁碟讀取資料，達到了近乎即時的響應速度。

- **Plotly Graph Objects (Go) 渲染引擎**： 不同於靜態的 Matplotlib，Plotly 提供基於 JavaScript (Plotly.js) 的互動式渲染。本系統深入利用了 Plotly 的 **Frames 模型**。
    - **客戶端播放邏輯**：系統預先在伺服器端計算好所有時刻的繪圖指令（Traces)，一次性將所有 Frames 推送到客戶端瀏覽器。當使用者點擊「播放」時，動畫的切換發生在瀏覽器的 GPU 渲染層，而非 Python 的主循環中。這有效解決了伺服器端重繪導致的畫面閃爍（Flashing）現象。
    - **SVG 與 WebGL 考量**：在向量場繪製中，每一幀可能包含數百個箭頭對象。Plotly 的高性能渲染確保了即使在高密集度資料下，縮放（Zooming）與平移（Panning）操作依然流暢。

2.5 UI 增強與客製化樣式工程（CSS Injection）

為了解決 Plotly 內建控制項（如播放按鈕、暫停按鈕）在特定佈局下無法自定義外觀的問題，系統實作了底層 **CSS 注入技術**。

透過 Python 字串注入 st.markdown 的方式，系統直接作用於瀏覽器的 DOM 結構。針對 Plotly 按鈕在 plotly_dark 樣板下顯得模糊且字體過粗的問題，我們自定義了無襯線字體（Arial/Helvetica）的渲染權重，並將按鈕背景色精確調整為白色煙霧色（White Smoke)，配合黑色細邊框。這不僅是美觀考量，更是為了確保工程人員在低光照環境下長時間觀測螢幕時，能清晰辨識控制狀態，降低視覺疲勞與操作失誤。

2.6 系統集成總結

綜上所述，本系統的技術棧並非簡單的工具堆疊，而是經過精密計算的解決方案。後端利用 Parquet 與 NumPy 壓榨硬體效能，前端透過 Streamlit 與動態 CSS 提升使用者交互體驗。這種「硬核資料處理」與「現代 Web 技術」的結合，構成了一個穩定、高效且具備高度專業性的分析平台。

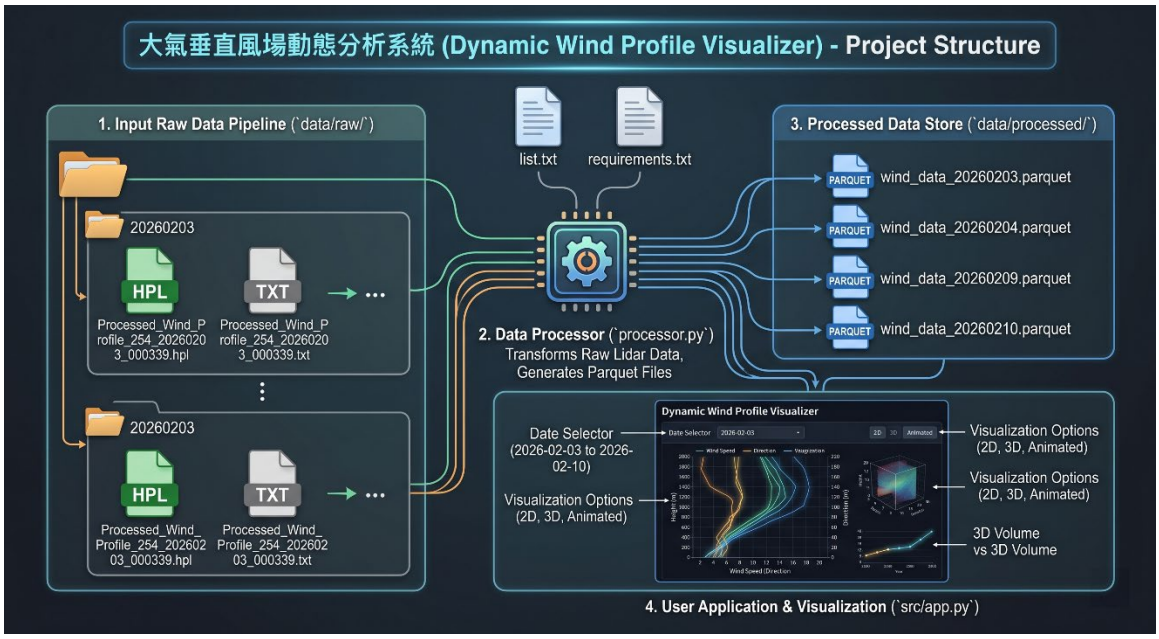

Figure 1. Folder structure

3. 後端數據處理模組 (Data Processing Module - Backend)

3.1 原始數據擷取與精確過濾 (Data Ingestion & Filtering)

後端引擎 WindDataProcessor (v1.2.2) 的首要任務是處理散亂於各個子目錄中的多普勒雷射雷達輸出檔案。在實際觀測環境中，系統會生成大量的 .hpl 檔案，且檔名中往往包含了站點資訊、時間戳與掃描模式。

為了確保數據處理的精確性，我們設計了一套基於 Regex (正規表達式) 與 Glob 模式的過濾機制。系統僅會選取符合 Processed_Wind_Profile_*.hpl 命名的核心數據檔案。這一層過濾有效排除了系統日誌、標頭檔或其他臨時緩存檔，確保了資料流水線（Data Pipeline）輸入端的高純度。透過 pathlib.Path.rglob 函數，系統可以遞迴掃描整個存儲架構，自動建立檔案索引清單。

3.2 時間序列解析與自動化日期分組 (Time Series & Date Grouping)

大氣分析往往是以「日」為基準單元進行觀測。然而，雷射雷達的單一檔案通常僅代表數分鐘的瞬時觀測。WindDataProcessor 實作了自動化的日期編組邏輯：

1. **檔名特徵提取**：利用正規表達式從檔名中提取 YYYYMMDD 格式的日期字串。
2. **字典映射映射**：建立一個以日期為 Key、檔案路徑列表為 Value 的映射字典。
3. **時序排序**：針對每一組日期內的檔案，根據檔名中的詳細時間戳進行排序，確保數據在垂直合併時不會出現時間順序錯亂（Time-step Shuffle）的問題。 這種自動

分組機制極大地簡化了使用者的負擔，使其無需手動分類原始檔案，即可直接產出以日為單位的結構化資料集。

3.3 數據格式標準化與 Parquet 轉換 (Serialization & Parquet Conversion)

數據讀取後，系統會執行一系列標準化的轉換步驟，這是確保物理計算準確度的關鍵：

- **Header 跳過與列名重定義**：原始 .hpl 通常含有數量不等的標頭行，處理器會精確定位數據起始行，並賦予具備物理含義的列名（如 height, speed, direction）。
- **無效值填充 (NaN Handling)**：氣象觀測常因信噪比（SNR）不足產生空值。系統將原始的 -99.9 或類似標識轉換為標準的 np.nan，以便後續繪圖引擎能正確處理斷點。
- **二進位序列化**：合併後的單日數據會以 df.to_parquet() 寫入。透過預設的 Snappy 壓縮與數據類型優化（例如將高度轉為 float32 以節省空間），單日資料的讀取速度在前端加載時達到了近乎「瞬發」的效果。

3.4 路徑工程與版本修復 (Path Engineering & Version Fixes)

在 v1.2.2 的更新中，我們針對跨平台運行時常見的技術債進行了重構：

- **SyntaxWarning 消除**：修復了早期版本中在 Docstring 內因錯誤使用轉義符號（如 \n 或路徑斜線）導致的 invalid escape sequence 警告，確保代碼符合最新的 Python 代碼風格標準。
- **標準化 Pathlib 整合**：針對 Windows 下的路徑長度限制與反斜線問題，所有檔案操作均不再使用字串拼接，而是透過 Path / "subdir" / "filename" 的運算子重載語法。這保證了程式碼在開發環境與生產部署環境（Production Environment）之間移動時，不會因環境變數或目錄結構差異導致路徑崩潰。
- **異常捕獲機制**：在 run_conversion 主流程中加入了對 FileNotFoundError 與 EmptyDataError 的精確捕捉，避免單一檔案損壞導致整個批次轉換任務中斷。

4. 前端動態視覺化儀表板 (Dynamic Visualizer & Dashboard - Frontend)

4.1 互動式介面設計與佈局 (UI/UX Design & Layout)

系統介面採用 Streamlit 的廣域佈局（Wide Layout），將操作邏輯與資料呈現進行空間解構：

- **側邊控制欄**：負責數據源選擇（Parquet 檔案切換）與全局物理參數設定。包含播放速度（ms）、觀測高度上限（m）、風速刻度（m/s）以及向量箭頭尺寸。
- **主展示區**：包含系統標題、動態 Plotly 圖表以及下方的「時刻進度條」。這種設計確保了核心繪圖區域擁有最大的視覺空間，符合專業工程監控的需求。

4.2 資料加載與時序快取機制 (Data Ingestion & Caching)

為了達成即時切換日期的流暢感，系統實作了多層資料處理邏輯：

- **高效能讀取**：利用 pd.read_parquet() 直接讀取二進位數據。由於 Parquet 的列式特性，即使單日數據量極大，載入過程依然極其迅速。
- **時序標籤化**：將載入的原始 timestamp 轉換為易讀的時分秒字串（%H:%M:%S），作為動畫幀的唯一索引標籤。
- **記憶體快取**：使用 @st.cache_data 裝飾器。當使用者在多個日期檔案間重複切換時，系統會直接從記憶體回傳數據，無需重新解析磁碟檔案，顯著降低了系統延遲。

4.3 動態繪圖引擎與 Frames 渲染機制 (Plotting Engine & Frames Rendering)

本系統的核心視覺化技術在於 Plotly 的動畫幀實作：

- **預計算策略**：程式碼中的 get_traces 函數負責生成單一時刻的所有繪圖元素（包含風速曲線與向量場）。系統會在啟動播放前，針對選定日期的所有時刻預先生成 go.Frame 物件列表。
- **客戶端渲染**：一旦所有幀生成完畢，整個 fig 物件會被一次性傳送到瀏覽器。播放過程中的幀切換完全由客戶端 JavaScript 驅動，不涉及伺服器端的 Python 重計算，從而徹底解決了傳統 Streamlit 動畫常見的「全網頁白屏刷新」問題。

4.4 向量場計算與物理標註實作 (Vector Calculation & Annotation)

為了在二維平面準確表達三維風場運動，系統實作了精確的幾何變換邏輯：

- **幾何投影轉換**：根據氣象學定義，風向（Direction）為風吹來的方向。系統將風速與風向轉換為水平分量（u, v），並考慮到圖表 Y 軸（高度）與 X 軸（風速）的長寬比差異（Aspect Ratio Fix），動態計算向量箭頭的偏移量（dx, dy）。
- **抽樣顯示策略**：為了避免大氣數據過於密集導致視覺混亂，向量箭頭採取「步進抽樣（Sampling by 4）」策略，在保持整體趨勢的同時維持介面清爽。
- **動態物理標註**：系統實作了「極大值追蹤」功能，每一幀都會動態計算當前剖面的最大風速點，並利用 go.Scatter 的 text 屬性在圖表上即時標註該點的高度與數值，提供直覺的極端事件觀測。

5. 版本演進與優化紀錄 (Version History & Optimizations)

5.1 數據處理模組的演進 (Evolution of Backend)

系統的開發遵循敏捷開發（Agile Development）模式，經歷了多次重大迭代：

- **v1.0 - v1.1**：初步實現了 HPL 檔案的讀取邏輯。此階段主要解決數據結構解析與 CSV 導出問題。

- **v1.2.1**：引入了 **精確檔案過濾系統**。針對原始數據中雜亂的標頭檔與日誌檔，定義了明確的 Processed_Wind_Profile_*.hpl 前綴識別，並實作了自動日期分組邏輯。
- **v1.2.2 (最新版)**：
  - **路徑標準化修復**：徹底轉向 pathlib 體系，修復了在 Windows 下因反斜線字串拼接導致的編譯警告與路徑崩潰問題。
  - **字串轉義優化**：修復了 Docstring 與註釋中 SyntaxWarning: invalid escape sequence 的語法錯誤，提升了代碼的嚴謹性。

5.2 視覺化儀表板的優化 (Evolution of Frontend)

前端介面經歷了從「靜態重繪」到「動態流暢」的轉型：

- **v2.x**：採用基本的 Streamlit 繪圖，但每切換一幀都會導致全頁刷新，視覺閃爍感嚴重，且無法滿足 10FPS 以上的流暢度需求。
- **v3.x**：導入 Plotly Frames 機制，將繪圖邏輯轉移至客戶端渲染，大幅提升了動畫的流暢度。
- **v3.2.2 (Final Visual Polish)**：
  - **CSS 樣式精煉**：為了解決 Plotly 內建按鈕在高對比樣板下字體模糊、邊框不明顯的問題，實作了精確的 CSS 注入。將按鈕背景設為白色煙霧（White Smoke），字體調整為無襯線（Sans-serif）並取消加粗，確保在不同解析度下皆能清晰辨識。
  - **佈局重構**：優化了時間進度條（Slider）與播放控制項的相對位置，解決了元件在寬螢幕顯示器上可能發生的重疊問題。

6. 結論與未來工作 (Conclusion & Future Work)

6.1 系統開發總結 (Conclusion)

本「大氣垂直風場動態分析系統」成功整合了後端資料工程與前端計算視覺化技術。透過 Parquet 列式存儲與向量化運算，系統成功克服了傳統 ASCII 資料解析緩慢的瓶頸。結合 Streamlit 與 Plotly 的動態渲染機制，為大氣邊界層研究提供了一個無需安裝複雜軟體即可運行的專業級分析平台。本系統不僅提昇了觀測數據的讀取效率，更透過動態視覺化揭示了許多靜態圖表難以捕捉的大氣物理特徵。

6.2 未來開發展望 (Future Work)

基於目前的穩定架構，未來可朝向以下方向擴展：

- **多維特徵提取**：開發自動化的低空噴流（LLJ）檢測算法，並在動態圖表上即時標繪噴流核心高度。
- **三維空間擴展**：整合多個雷射雷達站點的數據，從單站垂直剖面擴展至區域性的三維風場動態場模擬。
- **時頻分析整合**：引入小波變換（Wavelet Transform）等工具，分析風速脈動在不同時間尺度上的能量分布特徵。
- **雲端協作支持**：考慮將數據存儲遷移至雲端物件存儲（如 S3 或 Google Cloud Storage），實現跨地域的協同分析與數據共享。

附錄 (Appendix)

操作指令 (Commands)

請按照以下終端機指令順序執行數據處理與啟動視覺化系統。

1. 執行數據轉換 (Perform Data Conversion) 運行後端處理器，將原始 HPL 檔案轉換為結構化的 Parquet 檔案。

```
python src/processor.py
```

2. 啟動視覺化儀表板 (Launch the Visualization Dashboard) 運行前端 Streamlit 應用程式以查看處理後的數據。

```
streamlit run src/app.py
```

系統工作流程圖 (Workflow Flowchart)

下圖描述了數據從原始輸入到儀表板呈現的完整邏輯路徑：

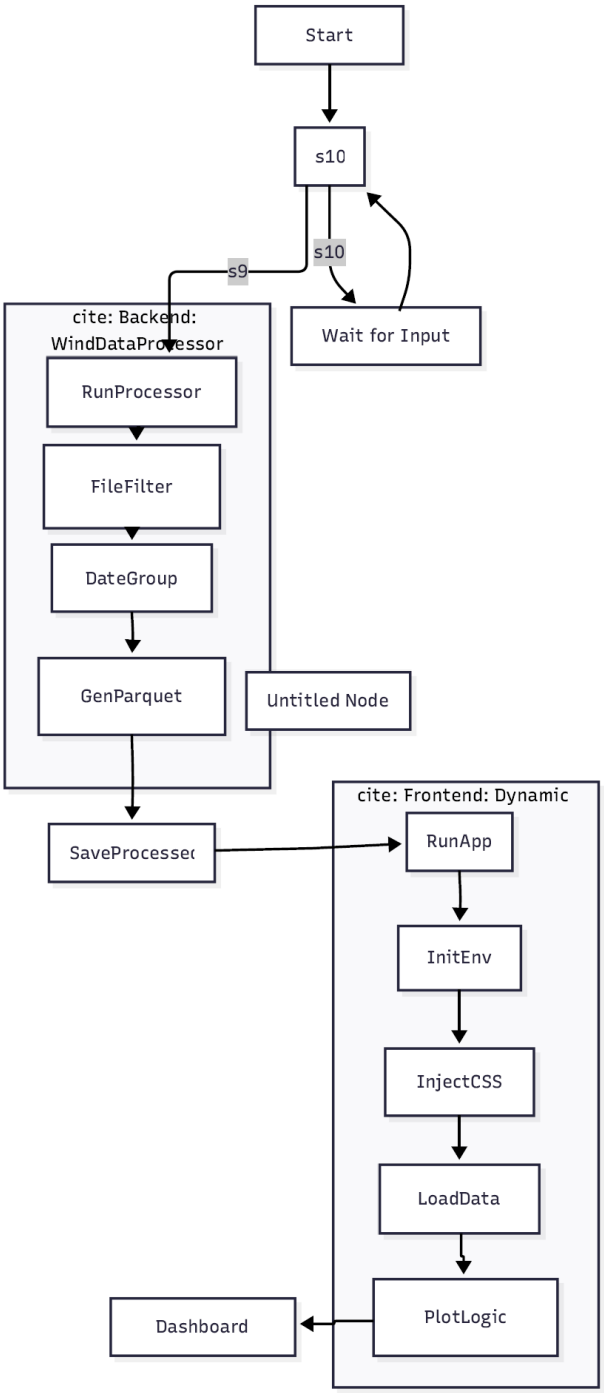

Figure 2. Data visualization procedure

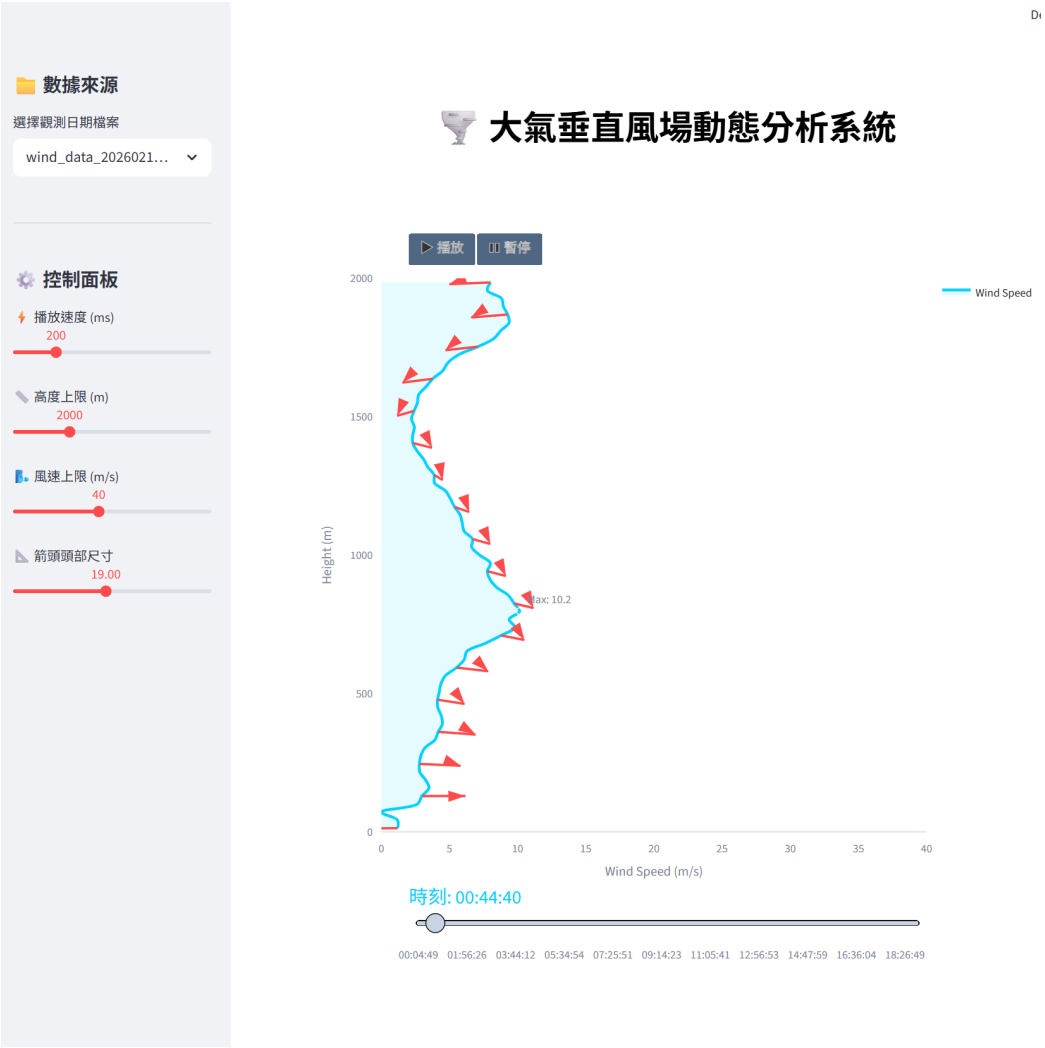

Figure 3. Snapshot of dynamic wind profile visualization UI